

abr. 29, 2026

Padrões arquiteturais

convidado Bruno Joaquim Cassia Dobler Fabio Diniz Francisco Schneider
Gabriel Severo Gabriel kohlrausch Ruan Rodrigues
aline.bianchini@t-systems.com ana.santos@t-systems.com
analista.kuehn@outlook.com anderson.hilario@t-systems.com
anderson.perim@t-systems.com andersongabriel_maria@hotmail.com
anselmogabriel421@gmail.com bmf.pedro@gmail.com
bruna.nery@t-systems.com bruno.destro@t-systems.com
caique.rodrigues@t-systems.com d.silva@t-systems.com
daniilosilva2017@hotmail.com desenvolvedor@tidsoftware.com.br
douglas.nishiyama@t-systems.com dpsnbsb79@gmail.com
eduardo.kubota@t-systems.com eudy.amparo@t-systems.com
fabio.marinelli@t-systems.com farley.t.i@hotmail.com
felipeneves089@gmail.com gustavo.ferrazfontes@gmail.com
hewerton.souza@t-systems.com i.parmigiani@t-systems.com
igor.romulo.santos@gmail.com jeanderson.silva@t-systems.com
jl.oliveir@gmail.com jonathan.jimenez@iatec.com karinetrevisann@gmail.com
l.regioli@outlook.com leandro.piqueira@t-systems.com
lucas.p.oliveira@outlook.pt lxsilvareis@gmail.com
marcelohcordeiro@gmail.com marcio@almob.com.br
marcos.ammon@gmail.com moraesguga@gmail.com n.segeti@t-systems.com
nadson.dr11@gmail.com Paulo Scatena rafael.dias@t-systems.com
regis.maioli@t-systems.com reinaldo.rodrigues@t-systems.com
renan-silva@t-systems.com renan.baptista.cabral@gmail.com
rodrigo_7sch@hotmail.com tatidornel@gmail.com
thiago.fernandes@t-systems.com victor.prospero@t-systems.com
victor.santos@t-systems.com victor.souza@t-systems.com
vinicius.bonicio@t-systems.com webdeveloperrm@gmail.com
wesley.boccaletto@t-systems.com wsilva@t-systems.com
fabiano.felgas@gmail.com Elemar Júnior

Anexos  Padrões arquiteturais

Registros da reunião  Transcrição  Gravação

Resumo

Especialistas discutiram arquitetura de software focada na redução de custos e riscos através do controle do acoplamento.

Engenharia focada em risco

O objetivo central da engenharia é reduzir custos e riscos de mudança, combatendo o acoplamento excessivo. Decisões arquiteturais devem priorizar a facilidade de evolução do software em vez de padrões dogmáticos.

Acoplamento e design intencional

O design deve ser intencional, equilibrando indireções e camadas conforme a necessidade do time e do negócio. Práticas como interfaces e padrões devem ser avaliadas por trade-offs, evitando complexidade desnecessária.

Contexto define a arquitetura

A escolha de modelos como Vertical Slice ou monólitos modulares depende do contexto da equipe e do tamanho do projeto. A senioridade reside em questionar fundamentos para evitar desperdícios técnicos.

Próximas etapas

- ☐ [Elemar Júnior] Postar Foto: Compartilhar foto da reunião.
- ☐ [O grupo] Analisar Logs: Obter logs de commits dos últimos 3 meses no Git. Verificar arquivos com maior número de alterações.
- ☐ [Farley Souza] Reestudar Patterns: Reestudar padrões de design focando em como eles lidam com acoplamento. Investigar relações de acoplamento com Repositório, Interface, Injeção de Dependência, Factory e Builder.
- ☐ [Elemar Júnior] Compartilhar Framework: Compartilhar framework de mentoria sobre design/arquitetura. O recurso está no site do livro.

Detalhes

- **Boas-vindas e Assuntos Iniciais:** Os participantes iniciaram a reunião com saudações e comentários sobre o "quase feriado" de amanhã. Foi mencionada a prática de "freezing de sistema" e a decisão de não realizar deploys inesperados amanhã para garantir uma sexta-feira tranquila ([00:00:00](#)).
- **Apresentação de Convidado e Tema Central:** Gabriel Kohlrausch apresentou Elemar Júnior como convidado especial, descrito como um guru de arquitetura e C-Sharp, para discutir elementos e conceitos de arquitetura relevantes para a carreira dos desenvolvedores ([00:01:46](#)). O objetivo é falar sobre o uso correto de C-Sharp ([00:03:08](#)).
- **Incentivo à Participação Ativa:** Elemar Júnior expressou satisfação em participar do programa de formação e fez uma provocação sobre o tema "reforço de peso" ([00:03:08](#)). Eles incentivaram a participação ativa dos presentes, sugerindo que abrissem a câmera para facilitar o diálogo e a troca de ideias ([00:04:24](#)).
- **Experiência e Carreira do Convidado:** Elemar Júnior se apresentou, mencionando ter 46 anos e 33 anos de experiência sendo pago para escrever código, começando aos 13 anos ([00:06:10](#)). Elemar trabalhou com C-Sharp desde sua primeira versão beta (cerca de 2001) e esteve próximo à equipe de linguagem, além de ter participado do desenvolvimento do banco de dados RavenDB ([00:07:13](#)).
- **Visão sobre Design e Boas Práticas (DDD):** O convidado mencionou que o conceito de "jeito certo" (em relação ao C-Sharp) nasceu de uma série no YouTube chamada "e DDD do jeito certo" ([00:07:13](#)). Programar em C-Sharp exige escrever um código que faça sentido, e a reunião visa discutir visões de design que vão além do C-Sharp ([00:08:28](#)).
- **Foco da Engenharia de Software: Redução de Custos e Riscos:** O objetivo fundamental da engenharia de software na Eximia é "reduzir o custo e o risco da mudança ao longo do tempo" ([00:09:42](#)). Boas práticas, padrões e decisões de design devem sempre contribuir para tornar as alterações de software mais baratas e fáceis, o que é o grande pilar do design ([00:10:51](#)).
- **Impacto da Mudança de Software no Negócio:** A importância de reduzir custos e riscos está no fato de que o software útil muda constantemente, refletindo mudanças em processos e na organização das empresas ([00:12:07](#)). Se o software é difícil de mudar, a organização perde

competitividade, pois alterações podem levar meses, atrasando a capacidade da empresa de se organizar e atender o mercado ([00:13:15](#)).

- **Justificativa para Práticas Técnicas:** Muitas práticas, como bons testes de unidade e o uso de Git para controle de versão, são adotadas para reduzir o risco e o custo da mudança ([00:13:15](#)). A IA também pode reduzir custos e riscos se for usada corretamente. A comunicação da necessidade de refatoração ou qualidade do sistema deve focar nas variáveis de custo e risco, que são as que o pessoal não técnico entende ([00:14:17](#)).
- **Dívida Técnica e Trade-offs de Arquitetura:** Márcio das Chagas Ramos compartilhou que está no curso para reestruturar seu software, que começou como um MVP e agora precisa de uma estruturação para o crescimento ([00:16:21](#)). A dívida técnica surge quando decisões ignoram o custo e o risco de mudar o software posteriormente ([00:17:26](#)). Um trade-off básico de arquitetura é priorizar o desenvolvimento rápido (reduzindo custo inicial) ou o custo de manutenção (que é vitalício); geralmente, vale a pena reduzir o custo de manutenção ([00:18:29](#)).
- **Acoplamento como Fator Crítico de Design:** O principal elemento de design que deve ser combatido para reduzir custo e risco é o acoplamento. Acoplamento e coesão são conceitos importantes, sendo o ideal software com baixa acoplamento e alta coesão ([00:19:44](#)) ([00:23:16](#)).
- **Desafio do Acoplamento em Múltiplos Projetos:** Wallace Silva descreveu o desafio de gerenciar um sistema com cerca de 120 projetos em C-Sharp, onde módulos comuns a todos os projetos (como "repository" e "model") estão crescendo ([00:20:52](#)) ([00:24:23](#)). Essa situação é um exemplo de dificuldade de gestão causada pelo acoplamento ([00:22:01](#)).
- **Tipos de Acoplamento (Eferente e Aferente):** Acoplamento é uma relação de dependência ([00:26:03](#)). O acoplamento eferente é a contagem de quantos elementos um componente depende (o serviço depende do repositório). O acoplamento aferente é a contagem de quantos elementos dependem de um componente (quantos dependem do repositório) ([00:27:17](#)).
- **Perigo da Mudança e Acoplamento:** Mudar um componente com alto acoplamento aferente (muita gente depende dele) é perigoso, pois pode quebrar o sistema inteiro. No caso de Wallace Silva, mudar o repositório é arriscado porque muitos projetos dependem dele ([00:30:26](#)).
- **Acoplamento e Estratégia de Testes:** O acoplamento é uma variável determinante para a estratégia de testes. Um alto acoplamento eferente (dependência de muitos) sugere a recomendação de testes de integração, enquanto um alto acoplamento aferente (muitos dependem dele) sugere a

recomendação de testes de unidade. Testes em componentes com alto acoplamento eferente são mais difíceis, exigindo muitos mocks ([00:31:50](#)).

- **BFF e Gestão de Acoplamento:** A justificativa para usar um Back-end for Frontend (BFF) é a gestão do acoplamento. O BFF reduz o acoplamento eferente do frontend para um, transferindo a complexidade para o BFF, tornando os testes mais fáceis ([00:34:15](#)). Testes de interface são mais caros e arriscados, pois são mais lentos e podem quebrar por mudanças de design ou automação ([00:35:30](#)).
- **Padrões de Projeto e Combate ao Acoplamento:** Quase todos os padrões de projeto nascem com o propósito de combater o acoplamento ([00:37:59](#)). A inversão de controle e injeção de dependência são formas de gestão do acoplamento ([00:34:15](#)).
- **Impacto das Interfaces na Dependência:** Criar interfaces para injeção de dependência pode reduzir o acoplamento, mas a interface se torna um ponto de alto acoplamento aferente ([00:39:04](#)). Mudar a interface fica mais difícil; se ela tiver implementações separadas, o acoplamento aferente dela aumenta, exigindo alterações em múltiplos lugares e gerando mais dores de cabeça ([00:41:04](#)).
- **Microserviços e Acoplamento de Banco de Dados:** A decisão de não usar um banco de dados por serviço em uma arquitetura de microserviços (monolito de banco de dados) faz com que o acoplamento aferente do banco de dados exploda, já que todos os microserviços dependem dele ([00:41:04](#)). Mudar o banco de dados se torna um pesadelo, aumentando o custo e o risco em vez de escalabilidade ([00:42:05](#)).
- **Combate ao Acoplamento e Boas Práticas:** A base de todas as práticas de engenharia de software (SOLID, testes, injeção de dependência, BFFs, microserviços) é a redução do acoplamento. Fazer o trabalho mecânico sem entender o combate ao acoplamento leva a decisões que aumentam o risco, mesmo parecendo estar seguindo as "melhores práticas" ([00:43:22](#)).
- **Acoplamento em Arquiteturas em Camadas:** A visão de acoplamento se aplica a designs em camadas (Presentation, Domínio, Infra) ([00:44:20](#)). O objetivo da separação (camadas) é possibilitar mexer em uma parte sem quebrar a outra ([00:45:36](#)).
- **Acoplamento de Mudança e Exagero de Camadas:** Farley Souza perguntou sobre a aplicação do acoplamento em monolitos com desenho em camadas ([00:44:20](#)). O problema é o acoplamento de mudança, onde uma alteração simples (como adicionar um campo na tela) exige mexer em muitos arquivos

(front-end, controller, DTO, serviço, entidade, repositório, banco de dados), o que não é eficiente ([00:46:37](#)).

- **Uso Incorreto de Padrões e Excesso de Projetos:** Criar um projeto específico para a implementação concreta de um repositório (por exemplo, para SQL Server) para não depender do banco é um exagero, visto que a troca de banco de dados é rara. O uso do padrão MediatR foi criticado por Elemar Júnior, que argumentou que, embora seja uma ferramenta incrível, o uso em contextos simples (como um cadastro de quatro campos) apenas complica o debug e abre mão da segurança do compilador em troca de detectar problemas em tempo de execução ([00:47:47](#)).
- **Gestão de Acoplamento e Escala do Projeto:** Eduardo Kubota questionou como justificar a refatoração e o desacoplamento de um monolito (ex: separar estoque e financeiro) para um time pequeno, quando cada mudança nova parece pequena demais para justificar o esforço de reestruturação ([00:50:56](#)). Elemar Júnior explicou que o design é profundamente influenciado pelo tamanho do time ([00:55:30](#)).
- **Impacto do Crescimento do Time:** Quando o time aumenta, a complexidade aumenta, pois várias pessoas começam a mexer nos mesmos arquivos, causando conflitos e dificuldades no pull request. Um projeto mal desenhado fica mais caro de manter à medida que o time cresce ([00:55:30](#)).
- **Como Nascem as Camadas e a Lei de Conway:** As camadas (frontend, backend, banco de dados) nascem como um ajuste natural para permitir que diferentes pessoas (ou times) trabalhem separadamente, por vezes levando à separação em diferentes repositórios ([00:56:48](#)). A Lei de Conway afirma que o jeito que o time é pensado (sua estrutura) determina a arquitetura do sistema ([00:57:50](#)).
- **Problemas de Dependência e Acoplamento Aferente Alto:** O acoplamento aferente alto em componentes compartilhados (como o banco de dados) faz com que todos os outros times dependam e sejam travados por mudanças nesse componente (o exemplo do DBA que é odiado porque todos dependem dele) ([00:57:50](#)).
- **Relação entre Arquitetura de Software e Estrutura Organizacional:** Elemar Júnior destacou que a solução de design adequada está diretamente ligada ao desenho da equipe e à estrutura organizacional. Uma equipe pequena pode gerenciar um monólito, mas uma equipe de 200 pessoas tentando gerenciar um monólito pode enfrentar dificuldades. A estruturação das equipes em "squads" (equipes trabalhando de ponta a ponta, ou *end to end*) é uma estratégia para diminuir o atrito e permite pensar em serviços em vez de apenas *front-end*, *back-end* e banco de dados ([00:59:02](#)).

- **Momento Certo para a Mudança Arquitetural:** É o momento de reavaliar a arquitetura quando o custo e o risco de mudança estão aumentando, o que geralmente ocorre com o crescimento ou alteração da equipe. Não é possível modificar a equipe sem alterar a arquitetura, e vice-versa, estabelecendo uma relação direta. A sugestão de Elemar Júnior para se preparar para a mudança, baseada em seus estudos, é o Domain-Driven Design (DDD) ([01:00:07](#)).
- **Reforço sobre o Domain-Driven Design (DDD) Estratégico:** Gabriel Kohlrausch reforçou que a menção é ao DDD estratégico, não à aplicação de conceitos como agregado ou repositório no sentido operacional que é frequentemente visto. Elemar Júnior desafiou a compreensão comum do que constitui um agregado, mencionando que a criação de um repositório sem o entendimento de um agregado é um problema frequente ([01:01:07](#)). A criação de uma interface de repositório apenas para injeção de dependência pode adicionar acoplamento desnecessário, especialmente se for uma interface genérica ([01:02:05](#)).
- **Diferença entre Design e Arquitetura:** Em resposta a uma pergunta de Eduardo Kubota, Elemar Júnior definiu *design* como o planejamento de elementos, responsabilidades e relacionamentos, como classes e suas dependências dentro de um projeto ([01:02:05](#)). *Arquitetura* é sempre sobre *design*, mas nem todo *design* é *arquitetura*. As decisões arquiteturais são aquelas que influenciam a capacidade de atingir os objetivos de negócio, respeitando restrições e atributos de qualidade, sendo decisões de design de mais alto nível ([01:03:11](#)).
- **Exemplos Práticos de Design vs. Arquitetura:** Decidir usar um repositório é uma decisão de *design*, pois tem pouca chance de impactar o atendimento do negócio ([01:03:11](#)). Em contraste, decidir sobre a inclusão de um *cache*, o banco de dados a ser usado, ou ter servidores de aplicação e *front-end* separados, são decisões de *arquitetura*. Decisões de *design* geralmente podem ser tomadas por uma única pessoa e são mais fáceis de reverter, enquanto decisões de *arquitetura* envolvem toda a equipe e são mais difíceis de alterar ([01:04:22](#)).
- **Análise de Alta Acoplamento em Order Service:** Ao analisar um exemplo de *Order Service*, Elemar Júnior observou quatro dependências explícitas no construtor, indicando um alto acoplamento. Gabriel Kohlrausch complementou que, frequentemente, isso leva a testes de integração que "mockam" (simulam) essas quatro dependências ([01:05:31](#)). Elemar Júnior criticou a criação de testes de unidade que apenas verificam se outros componentes foram chamados, o que não agrega lógica para validação ([01:06:27](#)).

- **Crítica à Criação de Interfaces Apenas para Teste:** A expectativa de substituir um componente, como um *gateway* de pagamento, ou a criação de interfaces unicamente para fins de teste, foi criticada. Elemar Júnior afirmou que se a única razão para criar uma interface é testar, o teste não é válido, pois aumenta o custo e o risco da mudança ao introduzir elementos de acoplamento de alta complexidade ([01:06:27](#)). O teste deve verificar o comportamento da unidade e não apenas se os métodos da interface foram chamados, evitando a busca por alto índice de cobertura sem valor prático ([01:07:51](#)).
- **Definições Corretas de Termos Técnicos:** Elemar Júnior corrigiu o vocabulário, enfatizando que se deve usar "teste de unidade" (em vez de "teste unitário") e "microserviço" com dois "S" ([01:08:41](#)). Outra correção é que o termo correto é "dívida técnica" (*technical debt*), e não "débito técnico," devido a um falso cognato ([01:09:51](#)).
- **Instabilidade do Sistema e Acoplamento:** Foi apresentada uma fórmula para testar o nível de instabilidade do sistema. Um componente com acoplamento aferente muito alto será mais "estável" (difícil de mudar), o que nem sempre é positivo ([01:09:51](#)). Gabriel Kohlrausch explicou que componentes estáveis são aqueles que não devem ser alterados, como um banco de dados, pois mudanças neles geram muita dor no sistema ([01:11:01](#)).
- **Objetivo de Compreender Dor e Acoplamento:** Os conceitos de acoplamento aferente e eferente são importantes para entender a origem das dificuldades, como a dificuldade em alterar um sistema. O objetivo não é paralisar a equipe, mas sim fornecer ferramentas para explicar a dor sentida no desenvolvimento ([01:12:23](#)). É desejável que o domínio seja mais estável, enquanto um *Handler* que interage com uma interface tende a ser mais instável ([01:13:32](#)).
- **Superando a Dificuldade dos Conceitos Avançados:** A dificuldade em discutir conceitos como acoplamento aferente e instabilidade está ligada à falta de familiaridade com essas ideias. A proposta é que a familiaridade substitua a dificuldade. O objetivo é compreender os fundamentos para um *design* bem-feito, que tem o potencial de transformar a compreensão sobre desenvolvimento de sistemas ([01:14:41](#)).
- **Acoplamento e Clean Architecture:** Elemar Júnior mencionou que o Clean Architecture, popularizado pelo Uncle Bob (Robert C. Martin), se concentra em combater o acoplamento. Ele sugeriu que se a essência do Clean Architecture não for entendida como combate ao acoplamento, o uso da arquitetura está incorreto ([01:16:07](#)). Elemar Júnior considera o Clean Architecture uma versão opinativa do *Ports and Adapters* ([01:29:45](#)).

- **Gerenciando Acoplamento Inevitável:** Farley Souza questionou como filtrar o acoplamento necessário daquele que é excessivo ([01:17:05](#)). Elemar Júnior respondeu que o acoplamento deve ser admitido apenas na medida exata da necessidade, sem inventar razões para tê-lo. A quantidade de acoplamento necessária está relacionada à forma como a equipe trabalha ([01:18:12](#)).
- **Revisão de Design Patterns sob a Perspectiva de Acoplamento:** Elemar Júnior sugeriu reestudar *design patterns* sob a perspectiva de como eles gerenciam o acoplamento, citando que a razão por trás de quase todas as boas práticas de programação é a gestão do acoplamento ([01:19:20](#)). Entender como um padrão lida com o acoplamento ajudará a determinar se o seu uso faz sentido no contexto de um código específico ([01:20:35](#)).
- **Estratégias de Flexibilidade com Instabilidade Controlada:** Gabriel Kohlrausch explicou que, quando um domínio estável precisa de alterações constantes (como regras de negócio ou cálculo de juros), isso gera dor devido ao acoplamento ([01:21:27](#)). Para componentes que precisam ser flexíveis e fáceis de alterar, eles devem ser tornados instáveis, e não estáveis ([01:22:29](#)).
- **Indireção como Solução para Acoplamento:** A criação de uma camada de indireção, como uma interface, pode receber o acoplamento, permitindo que o domínio principal permaneça estável. Essa interface, embora estável, possibilita a criação de regras de extensão instáveis e flexíveis, sem alterar o domínio ([01:23:38](#)). Este princípio se alinha ao *Strategy Pattern* ([01:25:01](#)).
- **O Perigo do Excesso de Indireção:** A lição de David Wheeler é que todos os problemas de ciência da computação podem ser resolvidos com um nível de indireção, exceto muitos níveis de indireção ([01:22:29](#)). O excesso de indireção, como o uso desnecessário de *mediators* ou interfaces de mensagens, pode tornar o sistema inteiro instável, aumentando o custo de alteração ([01:26:12](#)).
- **Código Superdimensionado vs. Código Errado:** Elemar Júnior enfatizou que um código com excesso de camadas de indireção (como serviços chamando *factories* que chamam repositórios) não está necessariamente errado, mas pode estar superdimensionado para o contexto. As decisões de design devem estar alinhadas com o problema que está sendo resolvido ([01:27:18](#)). Ele alertou que programadores que apenas copiam e colam código correm o risco de serem substituídos por IAs, pois o humano deve pilotar o jogo e entender o contexto ([01:28:30](#)).
- **Comparação entre Clean Architecture e Arquitetura Hexagonal (Ports and Adapters):** Clean Architecture é uma versão opinativa do *Ports and Adapters* (Arquitetura Hexagonal), compartilhando a regra da inversão de

controle e dependência para manter o domínio isolado ([01:29:45](#)) ([01:31:59](#)). O problema surge quando o Clean Architecture é assumido como a resposta universal, sendo a escolha errada para muitos problemas ([01:30:58](#)). Ambas as arquiteturas visam manter o *core* das entidades isolado ([01:33:38](#)).

- **Regras de Adoção de Estilos Arquiteturais (Clean, Vertical Slice, Hexagonal):** Igor Romulo questionou o melhor caminho para iniciar com diferentes designs ([01:51:50](#)). Elemar Júnior sugeriu que, para um projeto pequeno, mantido por um time reduzido, o Clean Architecture é uma abordagem funcional ([01:53:11](#)). O Vertical Slice é recomendado para times de tamanho médio que precisam de entregas *end to end* em uma base de código monolítica, facilitando a segmentação. A Arquitetura Hexagonal (Ports and Adapters) é mais adequada ao planejamento e implementação de microserviços ([01:54:28](#)).
- **Contexto Determina a Arquitetura Adequada:** Elemar Júnior ressaltou que a arquitetura adequada deve ser escolhida pelo contexto da aplicação. Por exemplo, um compilador não usaria Clean Architecture ou Vertical Slice, mas sim *pipes and filters* ([01:55:46](#)). O objetivo principal dessas abordagens, em geral, é reduzir o custo e o risco da mudança por meio do combate ao acoplamento ([01:34:47](#)).
- **Crítica à Implementação de Camadas Físicas Desnecessárias:** Gabriel Kohlrausch expressou sua opinião de que, no contexto atual, ele não vê a necessidade de iniciar um projeto com múltiplos *CS projects* (camadas físicas) ([01:57:04](#)). A separação física dos projetos pode não ser a melhor implementação, e é crucial que as interfaces vivam dentro de seus projetos para manter o domínio o mais isolado possível ([01:35:44](#)).
- **Isolamento do Domínio e Dívida Técnica Histórica:** Farley Souza descreveu uma implementação onde a interface do repositório está no domínio, mas a implementação fica na infraestrutura, um padrão comum em *templates* de DDD que tentam padronizar a abstração ([01:40:45](#)). Elemar Júnior questionou a necessidade de um projeto de infraestrutura separado, especialmente se a única razão for a hipótese de trocar de banco de dados, o que é raro e pode levar à perda de vantagens tecnológicas específicas do banco de dados em uso ([01:44:38](#)). Márcio das Chagas Ramos deu um exemplo histórico de quando a abstração do banco de dados era um requisito comercial nos anos 90, mas que raramente é o caso hoje ([01:48:34](#)).
- **A Regra do "Depende":** Quando confrontado com perguntas complexas como se uma estratégia de design é certa ou errada, Elemar Júnior e Gabriel Kohlrausch concordaram que a resposta é "depende". Eles sugeriram que se faça uma "pausa inteligente com semblante inteligente" antes de responder

(01:47:20). Elemar Júnior destacou que, embora o design possa fazer sentido em cenários específicos, na maioria dos casos, o pessoal se comporta como se fizesse sentido quando não faz (01:48:34).

- **Evitar Anti-padrões e a Organização Morfológica de Código:** Gabriel kohlrausch discutiu a necessidade de evitar anti-padrões, mencionando especificamente o risco de referências cíclicas entre camadas (01:57:57). Eles também sugeriram evitar a organização morfológica de pastas (como "pastinha de entes" ou "pastinha de interfaces"), optando por módulos orientados a contextos delimitados e subdomínios, o que é um passo melhor do que colocar tudo junto (01:59:05).
- **O Conceito de Interface e a Pasta "Interfaces":** Elemar Júnior afirmou que criar uma pasta chamada "interfaces" não é uma boa prática e demonstra uma falta de compreensão do conceito de interface em Orientação a Objetos. Ele explicou que a interface de um objeto compreende todos os seus membros públicos, e o artefato de interface em C\#ARP é um contrato que especifica os membros públicos que uma implementação se comprometerá a expor (02:00:22).
- **O Arrependimento no Design do C\#ARP e Implementação Explícita:** Elemar Júnior compartilhou uma conversa com Anders Hejlsberg (o designer do C\#ARP) sobre seu maior arrependimento no design da linguagem, que foi tornar a implementação de interfaces explícita (02:01:57). Essa necessidade de implementação explícita no C\#ARP, diferentemente do TypeScript, levou a "aberrações" e problemas de acoplamento, pois induziu as pessoas a criarem projetos de interface apenas para adicionar referências e realizar herança (02:04:44).
- **O Perigo da Abstração e a Regra do Repositório Único:** Gabriel kohlrausch e Elemar Júnior criticaram o dogma "sempre programe para uma abstração e nunca para uma implementação concreta", afirmando que o mau entendimento disso estragou o trabalho de muitos desenvolvedores (02:06:07). Elemar Júnior questionou a regra não oficial de que apenas o repositório pode acessar o banco de dados em implementações DDD, ilustrando como isso pode levar a operações ineficientes (como dar um `get all` e processar linha a linha em vez de usar um `UPDATE` direto no banco) (02:08:32) (02:10:36).
- **O Impacto das Boas Práticas na Performance e Custo de Mudança:** Elemar Júnior destacou que se uma suposta "boa prática" leva a um desempenho piorado ou atrapalha a evolução do sistema, ela não é uma boa prática. O objetivo final deve ser reduzir o custo e o risco de implementar mudanças, e

seguir regras dogmáticas sem contexto (como a interface genérica para repositórios) leva a projetos ruins ([02:14:14](#)).

- **O Custo de Cadeias de Chamada Desnecessárias e o Excesso de Padrões:** Gabriel kohlrausch e Elemar Júnior criticaram a criação de cadeias de classes ou camadas que apenas repassam dados sem adicionar valor, o que aumenta o custo de alteração ([02:16:52](#)). Eles também satirizaram a tendência de adicionar excesso de padrões de design, como DTOs, mappers, AutoMapper, Mediator, Result types, CQRS e event sourcing, apenas para parecer que estão seguindo as "melhores práticas" de internet ([02:17:50](#)).
- **O Design Intencional e a Redução de Acoplamento:** Elemar Júnior enfatizou que a provocação visa levar os desenvolvedores a um "design intencional", onde as decisões são tomadas para alcançar um resultado específico. Gabriel kohlrausch acrescentou que o desenvolvedor deve estar ciente do *tradeoff* (preço) de cada decisão; se o *tradeoff* não está claro, provavelmente não estão prontos para tomar essa decisão ([02:23:11](#)).
- **As Quatro Perguntas Fundamentais de Arquitetura:** Eduardo Kubota levantou o desafio de defender escolhas de design contra padrões impostos pela empresa ([02:24:35](#)). Elemar Júnior compartilhou quatro perguntas cruciais para orientar as decisões de design e arquitetura: a) Atende ao objetivo do negócio? b) Respeita as restrições (segurança, custo)? c) Está de acordo com os atributos de qualidade? d) Existe um jeito mais barato e menos arriscado de fazer a mesma coisa ([02:26:56](#))?
- **A Diferença entre Pleno e Sênior:** Elemar Júnior afirmou que a senioridade se manifesta na capacidade de responder às quatro perguntas de arquitetura e em entender que, às vezes, o caminho é fazer o que parece ser "errado" em favor de uma solução mais barata e menos arriscada ([02:30:30](#)). O sênior sabe fazer as perguntas certas no momento certo, enquanto o pleno sabe fazer tudo do jeito supostamente "certo" ([02:31:38](#)).
- **A Abordagem de Vertical Slice para Coesão:** Gabriel kohlrausch apresentou o *Vertical Slice* como uma forma de organizar o código, permitindo que funcionalidades passem apenas pelas camadas necessárias, ao invés de seguir um padrão rígido de passar por todas ([02:33:57](#)). Essa abordagem prioriza a coesão, permitindo, por exemplo, que funcionalidades simples usem apenas um CRUD direto no *end point* e funcionalidades complexas usem agregados e *handlers* ([02:35:28](#)) ([02:39:53](#)).
- **A Competição com a IA e a Necessidade de Pensamento Contextual:** Elemar Júnior alertou que se um desenvolvedor programa sem pensar no contexto e na intencionalidade, fazendo um trabalho mecânico e padronizado, eles estarão competindo diretamente com a IA ([02:36:44](#)). A IA consegue entregar

código padronizado e bem-feito, mas o valor do desenvolvedor está em entender o contexto do problema e pilotar a IA para garantir a melhor execução ([02:38:24](#)).

- **A Defesa do Monolito e a Arquitetura Lógica:** Gabriel kohlrausch apresentou uma estrutura de código onde um monolito é segmentado em módulos (como "pedidos" ou "financeiro") orientados a contextos delimitados, assumindo um pseudo recorte para facilitar uma futura segregação para microsserviços ([02:41:18](#)). Elemar Júnior reforçou que monolito não é sinônimo de código ruim e que é crucial diferenciar arquitetura física (deploy) de arquitetura lógica ([02:43:45](#)).
- ****O Uso de *Request Handler* (Mediator) e *Ports and Adapters***:** Gabriel kohlrausch explicou que o *handler* (Mediator) faz sentido quando uma funcionalidade tem múltiplas portas de entrada, como um *end point* e um *consumer* de fila. Esse uso intencional do *handler* é um exemplo de quando o acoplamento começa a fazer sentido, o que se alinha ao padrão *Ports and Adapters* ([02:46:09](#)).
- **Conclusão sobre a Motivação e o Tom da Discussão:** Elemar Júnior e Gabriel kohlrausch encerraram o encontro explicando que o tom exaltado foi intencional para provocar o debate e o pensamento crítico sobre padrões e arquitetura ([02:49:44](#)). O objetivo final é encorajar os participantes a desenvolverem um design intencional e a focarem em saber perguntar, em vez de apenas saber responder ([02:23:11](#)) ([02:50:57](#)).

Revise as anotações do Gemini para checar se estão corretas. [Confira dicas e saiba como o Gemini faz anotações](#)

*Como está a qualidade de **destas observações**? [Responda a uma breve pesquisa](#) para nos dar seu feedback, incluindo o quanto as observações foram úteis para o que você precisa.*